Ansible Service Account Implementation

Overview

This document details the implementation of a dedicated Ansible service account for enterprise-grade automation across the homelab infrastructure. The service account provides passwordless automation capabilities while maintaining security separation between personal administrative access and automated configuration management.

Implementation Objectives

Primary Goals

- Establish dedicated automation service account separate from personal accounts
- Enable passwordless automation workflows across all managed systems
- Implement enterprise-grade security practices and audit trail separation
- Provide scalable foundation for complex automation workflows
- Maintain compatibility with existing SSH configuration and VLAN architecture

Strategic Benefits

- **Security Separation**: Clear distinction between manual (nantwi) and automated (ansible) operations
- **Operational Excellence**: Eliminates password prompts during automation execution
- **Audit Trail**: Enhanced logging and accountability for automated changes
- **Enterprise Standards**: Mirrors production automation practices
- **Scalability**: Foundation for sophisticated automation workflows

Architecture Design

Account Structure

Account	Purpose	Access Level	SSH Key	Sudo Requirements
nantwi	Personal admin, development	Full administrative	`ansible-homelab-key`	Password required
ansible	Automation service	Automation tasks only	`ansible-homelab-key`	Passwordless (NOPASSWD)

Network Integration

- **VLAN Compatibility**: Works across all 6 VLANs (Management, BlueTeam, RedTeam, DevOps, EnterpriseLAN, Monitoring)
- **Tailscale Integration**: Remote automation via mesh VPN
- **pfSense Security**: Controlled by existing firewall rules
- **SSH Configuration**: Compatible with friendly hostnames (`wazuh-server`, `monitoring-server`)

Implementation Process

Phase 1: Service Account Bootstrap

Prerequisites Verification

```
# Verify SSH key exists
ls -la ~/.ssh/ansible-homelab-key*

# Test current connectivity
ansible all_in_one -m ping

# Check current configuration
ansible-config dump --only-changed | grep REMOTE_USER
```

Bootstrap Execution

```
# Run bootstrap playbook with current nantwi user
ansible-playbook ansible/playbooks/bootstrap-ansible-service-account.yml --ask-become-pass
```

Bootstrap Process: 1. **User Creation**: Creates `ansible` user on all managed systems 2. **SSH Configuration**: Installs `ansible-homelab-key` for service account 3. **Sudo Configuration**: Enables passwordless sudo (NOPASSWD: ALL) 4. **Dependency Installation**: Installs automation tools and utilities 5. **Verification**: Tests service account functionality

Expected Output

```
PLAY RECAP *********************************************************
192.168.10.2               : ok=15   changed=5    unreachable=0    failed=0
192.168.20.2               : ok=15   changed=5    unreachable=0    failed=0
192.168.60.2               : ok=15   changed=5    unreachable=0    failed=0
192.168.10.4               : ok=15   changed=5    unreachable=0    failed=0
```

Phase 2: Configuration Verification

Comprehensive Testing

```
# Run verification playbook
ansible-playbook ansible/playbooks/verify-ansible-service-account.yml
```

Verification Components: - User account creation and configuration - SSH key installation and permissions - Passwordless sudo functionality - Network connectivity and services - Package management capabilities - File system operations

Success Criteria

All verification tests should show checkmarks: - User Account: CREATED - SSH Keys: CONFIGURED - Sudo Access: WORKING - Network: WORKING - Automation: WORKING

Phase 3: Ansible Configuration Update

Configuration Changes

Update `/etc/ansible/ansible.cfg`:

```
# OLD CONFIGURATION:
remote_user = nantwi
ask_sudo_pass = true

# NEW CONFIGURATION:
remote_user = ansible
ask_sudo_pass = false
```

Implementation Steps

```
# Backup current configuration
sudo cp /etc/ansible/ansible.cfg /etc/ansible/ansible.cfg.backup

# Apply updated configuration
sudo cp ansible/ansible.cfg /etc/ansible/ansible.cfg

# Verify configuration
ansible-config dump --only-changed | grep -E "(REMOTE_USER|ASK_SUDO_PASS)"
```

Phase 4: Final Testing and Validation

Operational Testing

```
# Test basic connectivity with service account
ansible all_in_one -m ping

# Test privilege escalation (no password prompts)
ansible all_in_one -m shell -a "sudo whoami"
```

```
# Test cross-platform automation
ansible-playbook ansible/playbooks/install_htop.yml

# Test with friendly hostnames
ansible wazuh-server -m shell -a "systemctl status wazuh-manager --lines=2"
ansible monitoring-server -m shell -a "systemctl status grafana-server --lines=2"
```

Success Validation

- **No Password Prompts**: All automation executes without sudo password requests
- **Cross-Platform Support**: Playbooks work across Ubuntu and Rocky Linux systems
- **Service Management**: Can manage services across all VLANs
- **Friendly Hostnames**: Compatible with SSH configuration aliases

Security Implementation

Access Control Model

- **Principle of Least Privilege**: Service account limited to automation functions
- **SSH Key Management**: Shared key with appropriate access controls
- **Audit Trail Separation**: Clear distinction between personal and automated changes
- **Network Security**: Maintains existing VLAN isolation and pfSense controls

Security Benefits

- **Enhanced Accountability**: Clear separation of manual vs automated operations
- **Reduced Attack Surface**: Service account isolated from personal activities
- **Consistent Authentication**: Standardized access across all managed systems
- **Automated Security**: Foundation for security automation and compliance checking

Operational Capabilities

Current Automation Scope

The service account enables automation across: - **Wazuh SIEM Server** (192.168.20.2) - Security monitoring automation - **Monitoring Infrastructure** (192.168.60.2) - Grafana/Prometheus management - **Ansible Controller** (192.168.10.2) - Self-management capabilities - **Additional Systems** (192.168.10.4) - Extended lab infrastructure

Automation Examples

System Management

```
# Service control across infrastructure
ansible all_in_one -m systemd -a "name=ssh state=restarted"
```

```
# Package management
ansible all_in_one -m package -a "name=curl state=latest"

# Configuration deployment
ansible all_in_one -m copy -a "src=config.conf dest=/etc/myapp/ backup=yes"
```

Cross-Platform Operations

```
# Ubuntu-specific tasks
ansible ubuntu -m apt -a "name=nginx state=present"

# Rocky Linux-specific tasks
ansible rocky -m dnf -a "name=httpd state=present"

# Unified cross-platform automation
ansible-playbook site.yml
```

Security Automation

```
# Security updates
ansible all_in_one -m package -a "name=* state=latest" --limit ubuntu
ansible all_in_one -m shell -a "sudo dnf update -y" --limit rocky

# Configuration validation
ansible all_in_one -m shell -a "sudo audit-config.sh"
```

Advanced Automation Workflows

Planned Automation Development

Infrastructure as Code

- **Configuration Templates**: Standardized system configurations
- **Environment Provisioning**: Automated deployment of new services
- **Compliance Monitoring**: Automated security and compliance checking
- **Change Management**: Version-controlled infrastructure modifications

Security Automation

- **Automated Hardening**: Security configuration enforcement
- **Vulnerability Management**: Automated security updates and patching
- **Incident Response**: Automated response to security events
- **Compliance Reporting**: Automated generation of compliance reports

Operational Automation

- **System Monitoring**: Automated health checks and alerting
- **Backup Management**: Automated backup and recovery procedures
- **Performance Optimization**: Automated performance tuning
- **Capacity Planning**: Automated resource monitoring and scaling

Scalability Considerations

- **Additional Systems**: Framework supports easy addition of new managed systems
- **Complex Workflows**: Foundation for sophisticated multi-system orchestration
- **Role-Based Access**: Extensible to multiple automation service accounts
- **Integration**: Compatible with external automation tools and CI/CD pipelines

Monitoring and Maintenance

Service Account Management

```
# Monitor service account usage
ansible all_in_one -m shell -a "last ansible | head -5"

# Verify SSH key distribution
ansible all_in_one -m shell -a "sudo ls -la /home/ansible/.ssh/authorized_keys"

# Check sudo configuration
ansible all_in_one -m shell -a "sudo cat /etc/sudoers.d/ansible"
```

Automation Health Checks

```
# Daily automation health check
ansible all_in_one -m ping
ansible all_in_one -m shell -a "sudo systemctl is-system-running"

# Performance monitoring
ansible all_in_one -m shell -a "uptime"
ansible all_in_one -m shell -a "df -h | grep -v tmpfs"
```

Security Monitoring

```
# Audit automation access
ansible all_in_one -m shell -a "sudo journalctl -u ssh --since today | grep ansible"

# Monitor privilege escalation
ansible all_in_one -m shell -a "sudo grep ansible /var/log/auth.log | tail -5"
```

Troubleshooting Guide

Common Issues and Solutions

Issue: "Missing sudo password" errors

Cause: Bootstrap not completed or configuration not updated
Solution:

```
# Re-run bootstrap if needed
ansible-playbook bootstrap-ansible-service-account.yml --ask-become-pass

# Verify configuration
grep -E "(remote_user|ask_sudo_pass)" /etc/ansible/ansible.cfg
```

Issue: SSH connection failures

Cause: SSH key not properly distributed
Solution:

```
# Test SSH key access
ssh ansible@192.168.20.2 "whoami"

# Re-distribute keys if needed
ansible all_in_one -m authorized_key -a "user=ansible key='{{ lookup('file', '~/.ssh/ansible-homel
```

Issue: Permission denied errors

Cause: Incorrect file permissions or sudo configuration
Solution:

```
# Check file permissions
ansible all_in_one -m shell -a "sudo ls -la /home/ansible/.ssh/"

# Verify sudo configuration
ansible all_in_one -m shell -a "sudo cat /etc/sudoers.d/ansible"

# Fix permissions if needed
ansible all_in_one -m file -a "path=/home/ansible/.ssh mode=0700 owner=ansible group=ansible" --be
ansible all_in_one -m file -a "path=/home/ansible/.ssh/authorized_keys mode=0600 owner=ansible gro
```

Verification Testing

```
# Test complete automation workflow
ansible all_in_one -m ping
```

```
ansible all_in_one -m shell -a "sudo whoami"

# Verify service account functionality
ssh ansible@192.168.20.2 "sudo systemctl status wazuh-manager --lines=2"
ssh ansible@192.168.60.2 "sudo systemctl status grafana-server --lines=2"
```

Regular Maintenance Tasks

Daily Health Checks

```
# Verify service account connectivity
ansible all_in_one -m ping

# Check system status
ansible all_in_one -m shell -a "uptime"

# Monitor disk space
ansible all_in_one -m shell -a "df -h | head -5"
```

Weekly Security Audits

```
# Review recent ansible user activity
ansible all_in_one -m shell -a "last ansible | head -10"

# Check for unauthorized SSH keys
ansible all_in_one -m shell -a "sudo wc -l /home/ansible/.ssh/authorized_keys"

# Verify sudo configuration integrity
ansible all_in_one -m shell -a "sudo visudo -c -f /etc/sudoers.d/ansible"
```

Monthly Maintenance

```
# Review and rotate SSH keys if needed
ssh-keygen -t ed25519 -f ~/.ssh/ansible-homelab-key-new -C "ansible-homelab-$(date +%Y%m)"

# Update system packages via automation
ansible-playbook maintenance/system-updates.yml

# Generate access audit report
ansible-playbook reporting/access-audit.yml
```

Security Best Practices

Access Control

- Service account used exclusively for automation
- No interactive shell sessions for service account
- Regular review of automated tasks and their permissions
- Separation of duties between personal and automation accounts

Key Management

- Regular SSH key rotation (quarterly recommended)
- Secure storage of private keys with appropriate permissions
- Monitoring of key usage and access patterns
- Backup and recovery procedures for automation keys

Audit and Compliance

- Comprehensive logging of all automation activities
- Regular security audits and access reviews
- Documentation of all automated processes and their purposes
- Compliance with organizational security policies

Future Enhancements

Planned Automation Workflows

- Automated security hardening across all systems
- Centralized configuration management and drift detection
- Automated backup and disaster recovery procedures
- Integration with monitoring and alerting systems

Advanced Security Features

- Role-based access control for different automation functions
- Integration with external identity management systems
- Automated compliance checking and reporting
- Enhanced audit logging and forensic capabilities

Scalability Improvements

- Support for additional operating systems and platforms
- Integration with cloud-based infrastructure
- Container and Kubernetes automation capabilities
- CI/CD pipeline integration for infrastructure changes

Implementation Success Metrics

Technical Metrics

- 100% success rate for service account authentication across all systems
- Zero password prompts during automated operations
- Sub-second response times for automation connectivity tests
- 99.9% uptime for automation services

Operational Metrics

- Reduced manual administrative tasks by 80%
- Improved configuration consistency across all managed systems
- Enhanced security posture through automated compliance checks
- Accelerated deployment times for new services and configurations

Security Metrics

- Clear audit trail for all automated operations
- Separation of personal and automated access patterns
- Regular security assessments with automated remediation
- Compliance with enterprise security standards

Operational Best Practices

Regular Maintenance

- Monitor service account usage and access patterns
- Regular SSH key rotation (quarterly recommended)
- Audit automated task execution and permissions
- Review and update automation workflows

Security Monitoring

- Track service account login patterns
- Monitor privilege escalation usage
- Regular security audit of automation access
- Compliance with organizational security policies

Implementation Success

The Ansible service account implementation provides:

- **Passwordless Automation**: Eliminates password prompts during automation execution
- **Security Separation**: Clear distinction between personal and automated operations

- **Enterprise Standards**: Professional automation practices and audit capabilities
- **Scalable Foundation**: Ready for complex automation workflows and role-based architecture

This service account implementation establishes the foundation for the advanced role-based automation architecture documented in the next phase.

Ansible Service Account Status: Complete and Operational
Next Phase: Ansible Roles Architecture